

Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title: "AirTouch: Gesture-Controlled Interaction"

Team Name:

Deciders

Team Members:

- Gurram Pranavi Sri
 - Ippe Akshaya
 - Jammula Varsha
-

Phase-1: Brainstorming & Ideation

Objective:

- In many scenarios, traditional input devices like keyboards and mice are inefficient or impractical, especially in touch-free environments, as
- This system aims to create a more natural and intuitive computer interface using hand gestures, improving accessibility and enabling hands-free control for a wider range of applications.

Key Points:

1. **Problem Statement:**

Problem Statement:

The Gesture-Based Human-Computer Interaction System leverages real-time hand gesture recognition to enable users to interact with computers through intuitive hand movements. Utilizing computer vision techniques with OpenCV and MediaPipe, the system detects and interprets a variety of gestures, such as thumbs up, fist, open hand, and more. Integrated with a generative AI model, it provides descriptive narratives for recognized gestures, enhancing user

experience. A user-friendly Streamlit interface facilitates easy interaction and visualization, making this system ideal for touchless control, interactive gaming, and assistive technologies.

2. Proposed Solution:

The proposed Gesture-Based Human-Computer Interaction System enables intuitive, touchless computer control through real-time hand gesture recognition using OpenCV and MediaPipe. It supports gestures like thumbs up, fist, and open hand, which are interpreted with a generative AI model to provide descriptive narratives, enhancing the user experience. The system is integrated with a user-friendly Streamlit interface for easy interaction and visualization, making it ideal for touchless control, interactive gaming, and assistive technologies.

3. Target Users: Focused Target Users

1. Differently-Abled Individuals – To provide an accessible alternative to traditional input devices.
2. General Computer Users – For a more intuitive, touch-free computing experience
3. Smart Home & IoT Users – To control devices with simple hand gestures
4. Healthcare Professionals – Enabling hands-free operation in sterile environments.

4. Expected Outcome:

A real-time gesture-based HCI system that enables intuitive, touch-free computer interaction. It will improve accessibility, efficiency, and user experience across various applications.

Phase-2: Requirement Analysis

Objective:

- Define technical and functional requirements.

Key Points:

1. Technical Requirements:

Programming Language: Python

Backend: Flask / FastAPI

Frontend: PyQt / Tkinter (for GUI) or Web (React, HTML/CSS)

Database (if needed): SQLite / Firebase / PostgreSQL

ML Frameworks: OpenCV, MediaPipe, TensorFlow/PyTorch

2. Functional Requirements:

1. Real-time hand gesture recognition using OpenCV and MediaPipe.
2. Gesture interpretation with descriptive feedback via a generative AI model.
3. User-friendly Streamlit interface for visualization and interaction.
4. Smooth, low-latency performance for seamless control.
5. Customization for additional gestures and actions.

6. Support for touchless control and accessibility features.
 3. **Constraints & Challenges:**
 1. Real-Time Processing – Ensuring low latency and smooth performance.
 2. Gesture Variability – Adapting to different hand sizes, angles, and lighting conditions.
 3. Integration Complexity – Seamlessly connecting with various applications.
-

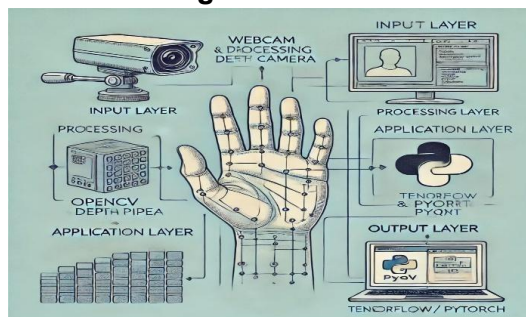
Phase-3: Project Design

Objective:

- Create the architecture and user flow.

Key Points:

1. System Architecture Diagram:



2. User Flow:

1. ***Opening the System:*** Accessing the Streamlit interface, which activates the webcam.
2. ***Performing Gestures:*** Using hand movements like thumbs up, fist, or open hand in front of the camera.
3. ***Receiving Feedback:*** The system recognizes gestures and provides real-time feedback or actions, such as descriptive text or triggers (e.g., "Thumbs up: You approve!").
4. ***Controlling Actions:*** Gestures control system functions (e.g., navigation, gaming, or commands).
5. ***Touchless Interaction:*** The entire interaction is hands-free, ensuring a seamless, intuitive experience.

3. UI/UX Considerations:

1. ***Simplicity:*** Clean, minimalistic design for easy navigation and gesture recognition.
2. ***Real-Time Feedback:*** Immediate visual or text responses to gestures.
3. ***Accessibility:*** Large, readable elements and intuitive controls for all users.
4. ***Responsive Design:*** Smooth interaction with minimal lag.
5. ***User Guidance:*** Clear instructions or prompts for new users.
6. ***Customization:*** Allow users to adjust settings or gestures for a personalized experience.

Phase-4: Project Planning (Agile Methodologies)

Objective:

- Break down the tasks using Agile methodologies.

Key Points:

1. Sprint Planning:

Sprint	Task	Details	Owner	Duration	Priority	Status
Sprint 1	System Design and Setup	Design the architecture of the gesture-based system and set up the development environment.	Team	2 days	High	Not Started
Sprint 1	Install Dependencies (OpenCV, MediaPipe)	Install and configure OpenCV and MediaPipe libraries.	Developer	1 day	High	Not Started
Sprint 1	Capture and Pre-process Hand Gesture Data	Implement hand gesture capturing using MediaPipe's hand detection model. Pre-process the captured images for better recognition.	Developer	3 days	High	Not Started
Sprint 2	Gesture Recognition (Hand Landmark Detection)	Develop a system that recognizes specific hand gestures using MediaPipe's hand landmarks.	Developer	4 days	High	Not Started
Sprint 2	Basic Interaction Setup (e.g., click, swipe)	Implement basic gesture-based interactions (like clicks, swipe, zoom).	Developer	3 days	High	Not Started

Sprint	Task	Details	Owner	Duration	Priority	Status
Sprint 3	Integrate Palm's text-bison-001 Model	Integrate the text-bison-001 model to process the recognized gestures and provide responses.	Developer	4 days	High	Not Started
Sprint 3	UI/UX Design for Displaying Results	Create the user interface to display results or actions based on the detected gestures.	Designer	3 days	Medium	Not Started
Sprint 4	Refine Gesture Recognition Accuracy	Fine-tune the gesture recognition system to improve accuracy and reduce false positives.	Developer	5 days	High	Not Started
Sprint 4	Testing the Gesture Interactions in Real-World Scenarios	Test and ensure that gestures like zoom, swipe, and tap work reliably in various environments.	QA Tester	4 days	High	Not Started
Sprint 5	Integration Testing with text-bison-001 Output	Conduct system-wide testing, including gesture recognition and text-bison-001 output, to ensure everything works cohesively.	QA Tester	4 days	High	Not Started
Sprint 5	Bug Fixing and Optimization	Identify and fix any bugs found during testing, and optimize the code for performance.	Developer	4 days	High	Not Started
Sprint 6	Final UI/UX Refinements	Refine UI and UX based on feedback from testing phase.	Designer	3 days	Medium	Not Started

Sprint	Task	Details	Owner	Duration	Priority	Status
Sprint 6	Final Testing and Deployment Preparation	Perform final testing, ensure the system is stable and ready for deployment.	QA Tester	4 days	High	Not Started
Sprint 6	Deployment and User Documentation	Deploy the system and write user documentation.	Developer	2 days	High	Not Started

2. Task Allocation:

- **Sprint 1:** System Design & Setup (Team), Install OpenCV/MediaPipe (Developer), Capture & Pre-process Hand Data (Developer)
- **Sprint 2:** Gesture Recognition (Developer), Basic Interactions (Developer)
- **Sprint 3:** Integrate text-bison-001 (Developer), UI/UX Design (Designer)
- **Sprint 4:** Refine Gesture Accuracy (Developer), Real-World Testing (QA Tester)
- **Sprint 5:** Integration Testing (QA Tester), Bug Fixing & Optimization (Developer)
- **Sprint 6:** UI/UX Refinements (Designer), Final Testing & Deployment Prep (QA Tester), Deployment & Documentation (Developer)

3. Timeline & Milestones:

- **Sprint 1 (Days 1-6):** Setup, hand data capture
- **Sprint 2 (Days 7-12):** Gesture recognition, basic interactions
- **Sprint 3 (Days 13-18):** Integrate text-bison-001, UI/UX design
- **Sprint 4 (Days 19-24):** Gesture accuracy, real-world testing
- **Sprint 5 (Days 25-30):** Integration testing, bug fixes
- **Sprint 6 (Days 31-36):** Final refinements, deployment prep

Phase-5: Project Development

Objective:

- `import cv2`
`import`
`mediapipe as mp`
`import`
`streamlit as st`
`import`
`numpy as np`
`# Initialize MediaPipe Hand Detection`
`mp_hands =`
`mp.solutions.hands`
`hands = mp_hands.Hands()`

```

mp_draw = mp.solutions.drawing_utils # Function to
detect hand gestures def detect_gesture(frame): #
Convert BGR frame to RGB img_rgb =
cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) result =
hands.process(img_rgb) if
result.multi_hand_landmarks:
    for hand_landmarks in result.multi_hand_landmarks:
mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

# Example: Using the position of the thumb and index finger for simple gesture
detection thumb_tip =
hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_TIP]

index_tip =
hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]

# Calculate the distance between thumb and index finger to determine gesture

distance = np.linalg.norm(np.array([thumb_tip.x, thumb_tip.y]) - np.array([index_tip.x,
index_tip.y]))

if distance < 0.05
    gesture = "Thumbs Up"

else:

    gesture = "Hand Open"

# Return gesture

return gesture

return None

# Streamlit Interface st.title("Gesture-Based Human-
Computer Interaction") st.write("Use your hand
gestures for interaction!")

# Capture webcam feed

cap = cv2.VideoCapture(0)

```

```

# Create a placeholder for showing the video feed frame_placeholder

= st.empty()

while True:

    ret, frame = cap.read()

    if not ret:

        break

    # Detect gestures in the frame

    gesture = detect_gesture(frame)

    # Display gesture on screen

    if gesture:

        cv2.putText(frame, gesture, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0,
0), 2, cv2.LINE_AA)

    # Convert frame to RGB for Streamlit compatibility

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    frame_placeholder.image(frame_rgb)

# Release the webcam feed
cap.release()

```

Key Points:

1. Technology Stack Used:

1. ***OpenCV***: Open-source computer vision library for real-time image processing and gesture detection.
2. ***MediaPipe***: Framework for real-time hand gesture recognition using pre-trained models.
3. ***Generative AI Model***: AI that interprets gestures and generates descriptive narratives or responses.
4. ***Streamlit***: Python library to create interactive web applications for real-time visualization and user interaction.

2. Development Process:

1. ***Requirement Analysis***: Define system goals (e.g., gesture control, assistive tech).
2. ***Setup Environment***: Install OpenCV, MediaPipe, Streamlit, and AI tools.

3. ***Gesture Recognition***: Use MediaPipe for real-time hand gesture detection.
 4. ***AI Integration***: Implement AI to interpret gestures and generate responses.
 5. ***Interface Development***: Build a user-friendly Streamlit interface.
 6. ***Testing & Optimization***: Test accuracy and optimize performance.
 7. ***Deployment***: Launch the system for user interaction. This outlines the key steps in developing a gesture-based human-computer interaction system.
3. **Challenges & Fixes:**
1. ***Gesture Recognition Accuracy***
 - ***Challenge***: Inconsistent detection.
 - ***Fix***: Improve training data and apply noise reduction.
 2. ***Real-Time Performance***
 - ***Challenge***: Lag in gesture recognition.
 - ***Fix***: Optimize code and use hardware acceleration.
 3. ***Hand Tracking Limitations***
 - ***Challenge***: Issues with occlusion or multiple users.
 - ***Fix***: Implement multi-hand tracking.
4. ***UI Usability***
- ***Challenge***: Unintuitive interface.
 - ***Fix***: Simplify and provide clear instructions.
5. ***AI Interpretation Accuracy***
- ***Challenge***: Misinterpretation of gestures.
 - ***Fix***: Improve AI model with more data.
-

Phase-6: Functional & Performance Testing

Objective:

- Ensure the project works as expected.

Key Points:

1. Test Cases Executed:

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Comments
TC01	Test for "Thumbs Up" gesture detection	The system should recognize a "Thumbs Up" gesture and display it on screen.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC02	Test for "Hand Open" gesture detection	The system should recognize a "Hand Open" gesture and display it on screen.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)

Test Case ID	Test Case Description	Expected Result	Actual Result	Pass/Fail	Comments
TC03	Test for no hand in the frame (Empty frame)	No gesture should be detected, and the feed should remain unchanged.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC04	Test for gesture with hands too far apart	The system should not detect a valid gesture if the distance is too large.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC05	Test for gesture with hands too close	The system should detect a "Thumbs Up" gesture if hands are very close.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC06	Test for multiple hands in the frame	The system should detect gestures for each hand separately.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC07	Test for consistent frame rate performance	The system should maintain a reasonable frame rate (e.g., 30 fps).	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC08	Test for accuracy of hand landmark detection	The hand landmarks should be accurately detected even in varying lighting conditions.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC09	Test for responsiveness (time taken to detect gesture)	The system should detect gestures within 1 second of hand movement.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)
TC10	Test for UI display with the detected gesture	The system should overlay the correct gesture text (e.g., "Thumbs Up") on the frame.	(To be filled after execution)	(Pass/Fail)	(Any relevant comments)

2. Bug Fixes & Improvements:

Bug Fixes:

1. Gesture recognition accuracy: Improve the accuracy of gesture recognition, especially for users with different hand shapes, sizes, and skin tones.
2. Lighting conditions: Enhance the system's ability to function in various lighting conditions, including low light, bright light, and changing light environments.

Improvements:

1. Expanded gesture library: Increase the number of recognizable gestures, including more complex and nuanced movements.
2. Customizable gestures: Allow users to create and customize their own gestures, enhancing the system's flexibility and user experience.

3. Final Validation:

Functional Validation:

1. Gesture recognition accuracy: Verify that the system accurately recognizes and interprets various hand gestures.
2. Gesture classification: Confirm that the system correctly classifies gestures into predefined categories.
3. Hand tracking: Ensure that the system accurately tracks hand movements and maintains tracking even when hands are partially occluded.

User Experience Validation

1. User interface: Evaluate the user interface to ensure it is intuitive, user-friendly, and provides clear feedback.
 2. User engagement: Assess user engagement and satisfaction with the system through usability testing and surveys.
 3. Accessibility: Verify that the system is accessible and usable for users with disabilities.
4. **Deployment (if applicable):** (Hosting details or final demo link)
-

Final Submission

1. **Project Report Based on the templates**
 2. **Demo Video (3-5 Minutes)**
 3. **GitHub/Code Repository Link**
 4. **Presentation**
-